

Python for Web Development

Lesson 5: Database Integration with SQLAlchemy

Database Integration with SQLAlchemy – Study Notes

Key Concepts

1. **SQLAlchemy ORM vs. Core** – Understanding the object relational mapping layer and when to use the lower level Core expression language.
2. **Model Definition** – Declaring tables as Python classes with `db.Model`, columns, relationships, and constraints.
3. **Migrations with Flask Migrate / Alembic** – Version controlling schema changes (create, alter, drop) safely in production.
4. **Session Management & Querying** – Using the scoped session (`db.session`) for CRUD operations, lazy/eager loading, and common query patterns.
5. **Integration with Flask** – Initialising the extension, configuring the database URI, and wiring the app factory pattern.

Summary

SQLAlchemy is the de facto ORM for Python and integrates tightly with Flask through the `Flask SQLAlchemy` extension. You start by configuring the database URI in your Flask app and creating a `SQLAlchemy` instance (`db`). Models are defined as subclasses of `db.Model`, where each attribute maps to a column, and relationships are expressed with `db.relationship` and foreign key columns.

Once the models exist, schema changes are managed with **Flask Migrate**, a thin wrapper around Alembic. Migrations are generated automatically (`flask db migrate`) and applied (`flask db upgrade`), allowing you to evolve the database without losing data.

Querying is performed through the session (`db.session`). The ORM provides a rich, expressive API for filtering, ordering, joining, and loading related objects. Understanding lazy vs. eager loading, pagination, and transaction handling is essential for building performant Flask applications that interact with relational databases.

Important Points

- **Configuration**

```
```python
```

```
app.config['SQLALCHEMY_DATABASE_URI'] = 'postgresql://user:pw@localhost/mydb'
```

```
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

```
db = SQLAlchemy(app)
```

```
```
```

- **Model basics**

- `id = db.Column(db.Integer, primary_key=True)` – primary key auto increment.

- ``db.Column(db.String(120), nullable=False, unique=True)`` – constraints.
- ``db.relationship('Child', backref='parent', lazy='dynamic')`` – one to many.
 - **Relationships**
- **One to One:** ``uselist=False`` in ``relationship``.
- **Many to Many:** Association table (``db.Table``) + ``secondary=`` argument.
 - **Migrations workflow**
- 1. ``flask db init`` – create migration folder.
- 2. ``flask db migrate -m "msg"`` – auto detect schema diffs.
- 3. ``flask db upgrade`` – apply to the DB.
- 4. ``flask db downgrade`` – roll back if needed.
 - **Session usage**
- Add: ``db.session.add(obj)``
- Bulk add: ``db.session.add_all([obj1, obj2])``
- Commit: ``db.session.commit()`` – persists changes.
- Rollback on exception: ``db.session.rollback()``.
 - **Common query patterns**
- ``Model.query.filter_by(name='Alice').first()``
- ``Model.query.filter(Model.age > 30).order_by(Model.created.desc())``
- ``Model.query.join(Other).filter(Other.status == 'active')``
- Pagination: ``Model.query.paginate(page, per_page, error_out=False)``
 - **Loading strategies**
- ``lazy='select'`` (default) – separate SELECT per access.
- ``lazy='joined'`` – eager load via JOIN.
- ``lazy='subquery'`` – eager load via sub query.
 - **App factory pattern**

```

python
db = SQLAlchemy()
def create_app(config_object):
    app = Flask(__name__)
    app.config.from_object(config_object)
    db.init_app(app)
    migrate = Migrate(app, db)
    return app

```

Examples

1p ã Defining Models & Relationships

```
python
```

models.py

```

from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()
class Author(db.Model):
    __tablename__ = 'authors'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(80), nullable=False, unique=True)
    # One to many: an author has many books
    books = db.relationship('Book', backref='author', lazy='dynamic')
class Book(db.Model):
    __tablename__ = 'books'
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(150), nullable=False)
    published = db.Column(db.Date, nullable=True)
    author_id = db.Column(db.Integer, db.ForeignKey('authors.id'), nullable=False)

```

Explanation: `Author` and `Book` are linked via `author\_id`. The `books` attribute on `Author` returns a query object (`lazy='dynamic'`) so you can further filter (`author.books.filter(...).all()`).

2b ã Running a Migration & Querying Data

```
``bash
```

Terminal

```

flask db init      # only once
flask db migrate -m "add author & book tables"
flask db upgrade

```

```
``python
```

usage in a view

```

from flask import Blueprint, jsonify, request
from .models import db, Author, Book
bp = Blueprint('api', __name__)
@bp.route('/authors/<int:aid>/books')
def author_books(aid):
    author = Author.query.get_or_404(aid)
    # eager load books to avoid N+1 queries
    books = Book.query.options(db.joinedload(Book.author)).filter_by(author_id=aid).all()
    return jsonify([{'title': b.title, 'published': b.published.isoformat() if b.published else None}
                    for b in books])

```

Explanation: After the migration, the tables exist. The view fetches an author and returns all their books. `joinedload` forces a single SQL query with a JOIN, preventing the “N+1” problem when

accessing `book.author``.

Quick Review

1. **What is the purpose of `Flask Migrate`` and which tool does it wrap?**
2. **How do you declare a many to many relationship in SQLAlchemy?**
3. **What is the difference between `lazy='dynamic`` and `lazy='joined``?**
4. **Write the three step command sequence to create and apply a new migration.**
5. **When should you call `db.session.rollback()``?**

Further Reading

- **Advanced Querying** – `with_entities``, subqueries, window functions.
- **Performance Tuning** – connection pooling, statement caching, profiling SQL.
- **Testing with SQLAlchemy** – using `sqlite:///memory:`` and transaction rollbacks.
- **Asynchronous Support** – `SQLAlchemy 2.0`` async engine with Flask (Quart/ASGI).
- **Database Specific Features** – PostgreSQL JSONB, full text search, and GIS extensions.

